my handwritten notes

N8N AUTOMATION FROM ZERO

What it is, what it costs, whether it's worth it, and how to build real workflows step by step

beginner friendly · real workflows · AI agents · costs & licence · Q&A

Running example: PeakFit, a small chain of three gyms (Aachen, Cologne, Düsseldorf) that wants less copy-paste and more time for members.

CONTENTS

Getting oriented

What Is Automation? 1
Meet n8n 2
What People Use n8n For 3
n8n vs Zapier vs Make vs Power Automate 4
What Does n8n Cost? 5
The Licence in Plain English 6
Is It Worth Learning? 7

Getting started

Three Ways to Start 8
Tour of the Editor 9
Inside a Node 10
The Core Vocabulary 11

Building blocks

Triggers: How Workflows Start 12
How Data Flows: Items 13
Expressions: Making Fields Dynamic 14
Expressions Cookbook 15
Your First Workflow, Step by Step 16
Core Nodes You'll Use Daily 17
Logic: If, Switch, Filter 18
Combining Data: Merge, Aggregate, Split Out 19
Loops, Batches & Waiting 20
Talking to Any App: HTTP Request & APIs 21
Webhooks & Forms 22
The Code Node 23
Credentials & Security 24
When Things Break: Error Handling 25
Debugging Like a Pro 26

AI & advanced

AI in n8n: The Building Blocks 27
Build an AI Inbox Assistant 28
Inbox Assistant: Test & Tune 29
AI Builder, Assistant & MCP 30
Sub-workflows: Build Once, Reuse Everywhere 31

Real projects

Project 1: Rain Alert for Outdoor Classes 32
Project 2: Trial Form to CRM 33
Project 3: Weekly PMO Status Report with AI 34

Going further

Self-Hosting Without Fear 35
Best Practices for Clean Workflows 36
Shortcuts & Speed Tips 37
Your 30-Day Learning Plan 38

Review

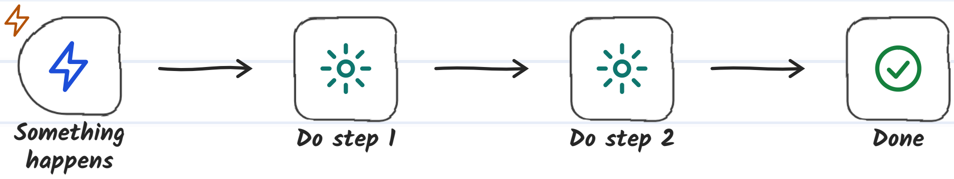
Q&A: Doubts You'll Probably Have (1) 39
Q&A: Doubts You'll Probably Have (2) 40

Getting oriented

01 What Is Automation?

The idea in one sentence

When **something happens**, do **these steps** for me, every time, without me clicking.



The shape of every automation

Manual vs automated: a PeakFit Monday

By hand (45 min every week)

1. Open the sign-up sheet
2. Filter last week's new members
3. Count them per studio
4. Type an email to the 3 studio managers
5. Paste the numbers, hit send
6. Forget one week, get asked "where's the report?"

Automated (0 min)

1. Monday 08:00 the workflow wakes up
2. Reads the sheet, filters last 7 days
3. Counts per studio
4. Emails the managers
5. Posts the totals in Slack
6. Never forgets, never typos

What makes a task worth automating?

Ask yourself	If yes...
Do I do it often (daily, weekly)?	big win over time
Does it follow the same steps each time?	easy to automate
Do I copy data between two apps?	classic automation job
Do mistakes happen when tired or rushed?	automation is consistent
Does it need human judgement every time?	automate the prep, keep the decision

Tip

A good first candidate: anything you'd describe as "boring, repetitive, and annoying when I forget it".

☆ Key Takeaway ☆

Automation = trigger (when) + actions (what). Everything in n8n is built from that pattern.

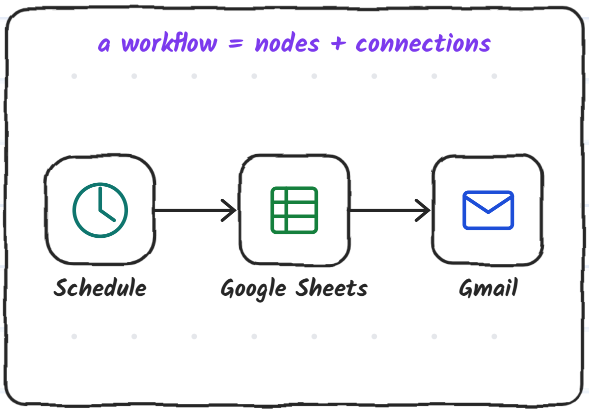
Automate the boring parts, keep the thinking parts.

02 Meet n8n

What it is

n8n (say "n-eight-n", short for *nodemation*) is a **workflow automation tool**. You connect blocks called **nodes** on a canvas, and each node does one job: read a sheet, send an email, call an AI model, decide yes/no.

- Started in **Berlin in 2019**
- **Visual** like Zapier, but you can add **code** when you want
- **Self-host it for free** or use their cloud
- Hundreds of ready-made app integrations, plus **any app with an API**
- Strong **AI agent** features built in



Why people pick n8n

Visual + code
Drag and drop for the easy 90%, a Code node for the tricky 10%.

You can own it
Run it on your own laptop or server, so data can stay in-house.

AI-native
AI Agent, chat, memory and tools are normal nodes on the canvas.

Fair-code, not classic open source
The code is public and self-hosting is free under n8n's **Sustainable Use License**. There are limits on reselling n8n itself (see the licence page).

☆ Key Takeaway ☆

n8n = a canvas where you chain nodes into workflows. Visual by default, code when needed, self-hostable, strong on AI.

Automate the boring parts, keep the thinking parts.

Getting oriented

03

What People Use n8n For

Real, common workflows grouped by role. ☆ = great first project.

Area	Workflow idea	Typical apps
Reporting ☆	weekly numbers from a sheet or database, emailed or posted to Slack	Sheets, Postgres, Gmail, Slack
Lead handling ☆	website form → CRM → welcome email → sales notified	Forms, HubSpot, Pipedrive, Gmail
PMO / projects	overdue tasks reminder, weekly status digest, new risk → ticket	Jira, Asana, Notion, Teams
Data syncing	keep a CRM, a sheet and a database in sync	Airtable, Sheets, MySQL
Data analysis prep	pull data from APIs nightly, clean it, store it for dashboards	HTTP Request, BigQuery, Power BI dataset
Inbox & support	classify emails with AI, draft replies, route tickets	Gmail, Outlook, Zendesk, OpenAI
Documents	extract invoice fields from PDFs, save to accounting sheet	Drive, Extract from File, AI
Monitoring	check a website/API every 5 min, alert if down	HTTP Request, Telegram
Content	turn a blog RSS feed into social posts for review	RSS, AI, LinkedIn
HR / onboarding	new hire → accounts, checklist, welcome message	BambooHR, Google Workspace
AI agents	chat assistant that answers from company docs	Chat Trigger, AI Agent, vector store

Your profile: Business / Data Analyst + PMO

Report robot

Every Monday, pull KPIs and email a summary with the 3 biggest changes.

Tracker nanny

Daily check of the project tracker, remind owners of overdue tasks.

Clean data pipe

Nightly API pull, clean with code, append to a database your dashboard reads.

☆ Key Takeaway ☆

If data moves between apps on a schedule or after an event, n8n can probably do it.

Getting oriented

04

n8n vs Zapier vs Make vs Power Automate

	n8n	Zapier	Make	Power Automate
Style	node canvas, visual + code	simple step list	visual scenario canvas	flows, Microsoft-centred
Self-host	yes, free (Community)	no	no	no (cloud, plus desktop RPA)
Billing unit	one execution = a whole run	per task (each action step)	per operation / credit (each module run)	per user / flow licence
Code	JavaScript & Python in-node	limited code steps	limited	expressions, some code
AI agents	built-in agent, memory, tools, MCP	AI steps & agents	AI modules & agents	Copilot / AI Builder
Learning curve	medium	easiest	medium	medium
Best for	technical teams, complex logic, data control	quick no-code automations	visual power users, good value	companies living in Microsoft 365

The pricing difference, made concrete

A workflow with **10 steps** runs **500 times** a month:

✓ **n8n**

500 executions (the whole run counts once, however many steps)

⚠ **Step-counting tools**

roughly **5,000 tasks / operations** (every step counts)

Good to know

Exact plan prices change often. Compare on each vendor's pricing page before deciding, and count how **your** tool bills: by run, by step, or per user.

Tip

Many companies already pay for Power Automate through Microsoft 365. Knowing n8n still helps: the thinking (triggers, JSON, APIs, branching) transfers to every tool in this table.

☆ **Key Takeaway** ☆

n8n wins on control, flexible logic and price for many-step workflows. Zapier wins on simplicity, Power Automate on Microsoft integration.

Getting oriented

05

What Does n8n Cost?

Option 1: n8n Cloud (they host it)

Prices from n8n's pricing page (billed annually; paying monthly costs a bit more):

Plan	Price	Executions / month	Good for
Starter	€20 / month	2,500	learning, personal & small team use
Pro	€50 / month	10,000	busy teams, more AI credits, workflow history
Business	€667 / month	40,000	companies needing SSO, Git, environments (self-hosted)
Enterprise	on request	custom	large orgs, SLAs, compliance

- All plans: **unlimited users and workflows**. Small startups can get a discount on Business.
- **Execution** = one complete run of a workflow. Number of steps or data size doesn't matter.

Option 2: Self-host the Community Edition

€ Licence

€0. Free for your own use and your company's internal use.

Server

Your laptop (€0, only while it's on) or a small cloud server, very roughly **€5 to €15 / month**.

Your time

Updates, backups, security. This is the real cost.

Hidden costs to plan for

- **AI usage**: OpenAI, Anthropic, Google etc. bill per use through your own API key (cloud plans include some AI credits).
- **Other apps' plans**: the CRM or email tool you connect may need a paid tier for API access.
- **Executions creep**: a workflow every 5 minutes = about **8,640 runs a month**. Check the schedule before picking a plan.

€ Cost check

Learning costs nothing: self-host on your laptop with Docker, or use the cloud free trial, then decide.

☆ Key Takeaway ☆

Cloud = pay for convenience (from €20/month). Self-host = free licence, but you are the IT department.

Getting oriented

06 The Licence in Plain English

n8n's **Sustainable Use License** ("fair-code") applies to the self-hosted Community Edition. Summary of n8n's own FAQ:

Allowed

- Running your own business processes on n8n
- Personal projects, learning, research
- Building automations **for clients** on your instance, as long as clients can't build or edit them
- Charging for building, setup, maintenance, training and consulting
- Using n8n behind the scenes in your product, where users only trigger automations **you** built
- Running several instances

Not allowed

- Hosting n8n **as a service** where customers build workflows
- Letting external users build or configure workflows through your product (custom UI, API, MCP or an AI agent)
- Forking n8n to launch your own automation product
- White-labelling (removing n8n branding)
- Using enterprise features without an enterprise licence

Quick test

Situation	OK?
You automate PeakFit's reporting on a self-hosted n8n	yes
A consultancy builds workflows for 5 clients, clients only see results	yes
You sell an "automation platform" where customers drag their own nodes	no, needs a commercial licence
You run a paid workshop where people practise on your n8n for a day	yes (time-limited)

Watch out

This is a summary, not legal advice. For anything commercial or grey, read the licence FAQ or ask n8n at license@n8n.io.

☆ **Key Takeaway** ☆

Using n8n to run a business = fine. Selling n8n itself as a workflow-building service = not fine without a deal.

Getting oriented

07 Is It Worth Learning?

Honest pros and cons

Worth it because

- Automation and **AI agents** are among the most requested skills in 2025/26 job ads for analysts, ops and PMO roles
- Teaches transferable thinking: **APIs, JSON, logic, error handling**
- Saves hours every week straight away
- Free to learn, and your projects double as portfolio pieces
- Bridges business and tech: exactly where a BA sits

Think twice if

- Your company only allows Microsoft tools (learn **Power Automate** too)
- You never want to see JSON or a bit of code
- You'd self-host but nobody will maintain the server
- The task is a one-off (just do it by hand)

Verdict by situation

You are...	Verdict
A BA / DA / PMO wanting to stand out	☆ **yes**, build 2 to 3 portfolio workflows
Doing lots of reporting and data copying	**yes**, quick payback
Interested in AI agents	**yes**, one of the most practical ways to build them
Purely non-technical, simple 2-step needs	Zapier may be enough
In a strict Microsoft-only company	Power Automate first, n8n for personal skill

☆ Pro move

In interviews, "I automated our weekly status report with n8n and saved ~3 hours a week" beats "I know automation tools". Measure your time saved.

> *Learn the concepts once (trigger, items, expressions, branching, APIs) and every automation tool becomes easy.*

☆ Key Takeaway ☆

For an analyst or PMO profile in 2026, yes: n8n is worth learning, especially with its AI agent features.

Getting started

08 Three Ways to Start

Way	Cost	Setup time	Best for
n8n Cloud	free trial, then from €20/month	2 minutes	trying fast, no tech setup, webhooks work from anywhere
Docker on your laptop	€0	10 to 15 minutes	learning for free, full control
npx (Node.js)	€0	5 minutes	quick test if you already have Node.js

Option A: Cloud

1. **Sign up** at n8n.io and start the trial
2. **Name** your instance (you get yourname.app.n8n.cloud)
3. **Open** the editor and create a workflow

Option B: Docker (recommended for free learning)

1. **Install** Docker Desktop (Windows / Mac) and start it
2. **Create** a volume so your work is saved
3. **Run** the n8n container
4. **Open** localhost:5678 and create the owner account

```
docker volume create n8n_data
docker run -it --rm --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n
docker.n8n.io/n8nio/n8n
```

Option C: npx

```
npx n8n
```

Needs a recent LTS version of Node.js. Then open **http://localhost:5678**.

⚠ **Watch out**

On your laptop, n8n only runs **while the laptop is on and the terminal/Docker is running**. Schedules won't fire at 08:00 if your laptop is asleep, and outside apps can't reach your webhooks (they'd need a public URL).

Tip

The `-v n8n_data:...` part is what keeps your workflows and credentials when the container stops. Without it, you lose everything on restart.

☆ **Key Takeaway** ☆

Learn on Docker for free, move to Cloud or a small server when a workflow must run 24/7.

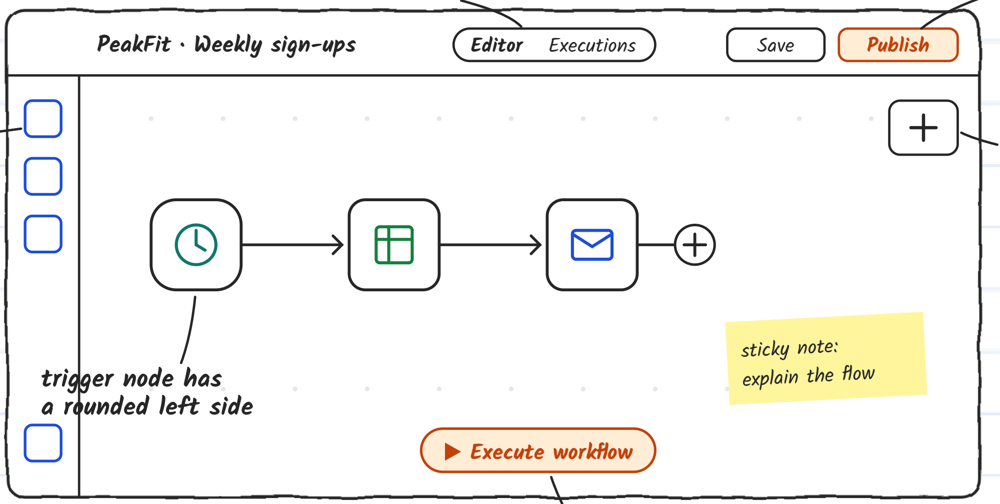
09 Tour of the Editor

Editor / Executions

Publish = make it live

Left bar
home, templates, credentials, settings

Nodes
panel: + or the Tab key



trigger node has a rounded left side

Run it now (test)
does not need Publish

What each area does

Area	Use it to
Canvas	place nodes and connect them (drag from a node's right dot to the next node)
Nodes panel (+ or Tab)	search 400+ nodes: apps, triggers, logic, AI
Execute workflow	run a test right now, with results shown on every node
Executions tab	history of every run (when, success/error, data at each step)
Save / Publish	Save stores a draft; **Publish** makes the live version run on its triggers (n8n 2.0; older versions used an "Active" toggle)
Overview (left bar)	all workflows, credentials and executions in one place
Templates	ready-made workflows to copy and adapt

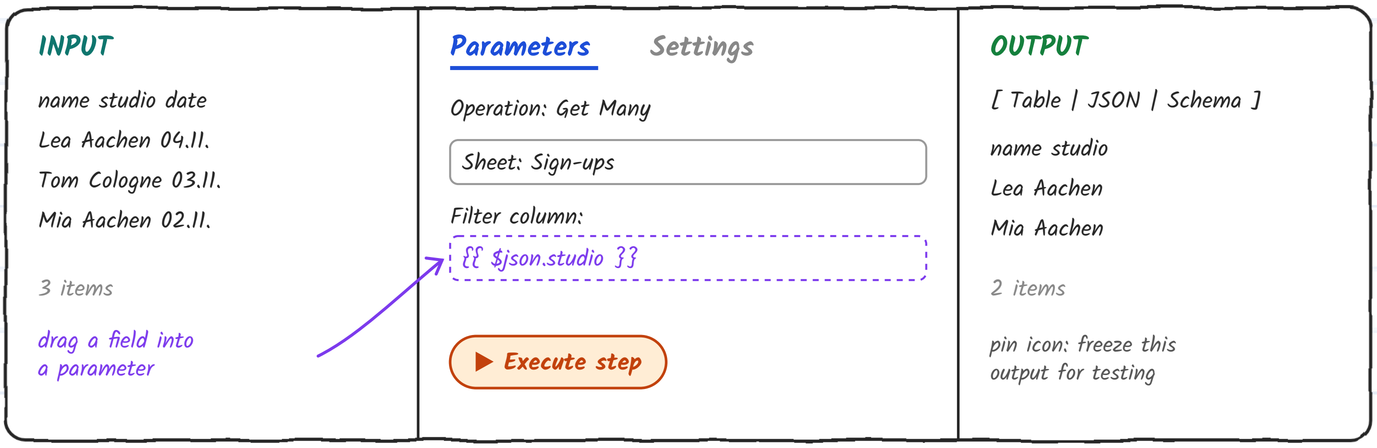
☆ Key Takeaway ☆

Build on the canvas, test with Execute workflow, check the Executions tab, then Publish to go live.

Getting started

10 Inside a Node

Double-click any node to open its **detail view**. This is where you spend most of your time.



INPUT

name	studio	date
Lea	Aachen	04.11.
Tom	Cologne	03.11.
Mia	Aachen	02.11.

3 items

drag a field into a parameter

Parameters Settings

Operation: Get Many

Sheet: Sign-ups

Filter column:

`{{ $json.studio }}`

▶ Execute step

OUTPUT

[Table | JSON | Schema]

name	studio
Lea	Aachen
Mia	Aachen

2 items

pin icon: freeze this output for testing

The three panels

- **Input**: data arriving from the previous node
- **Parameters**: what this node should do
- **Output**: what it produced (view as **Table**, **JSON** or **Schema**)

The Settings tab

- **Always Output Data**: continue even with 0 results
- **Execute Once**: only use the first item
- **Retry On Fail**: try again if an app hiccups
- **On Error**: stop, continue, or send to an error output
- **Notes**: write what the node does

Tip

Execute step runs just this node with the data it already has. You test one step at a time instead of the whole workflow.

☆ Key Takeaway ☆

Input on the left, settings in the middle, output on the right. Read every node left to right.

Getting started

11

The Core Vocabulary

Word	Means	PeakFit example
Workflow	a set of connected nodes that does one job	"Weekly sign-up report"
Node	one step (an app action, logic, code, AI)	Google Sheets: get rows
Trigger	the node that starts a workflow	every Monday 08:00
Action node	does something: read, create, send, update	send a Gmail
Connection	the line between nodes; data flows along it	Sheets → Gmail
Item	one record travelling through the workflow	one new member
JSON	the text format items are stored in	`{ "name": "Lea" }`
Expression	a dynamic value inside `{{ }}`	`{{ $json.name }}`
Credential	saved login / API key for an app	PeakFit Gmail account
Execution	one run of the workflow	Monday's run at 08:00
Test vs production	manual test runs vs live runs from triggers	Execute workflow vs Published
Pinned data	frozen sample output used while building	3 fake sign-ups
Sub-workflow	a workflow called by another workflow	"Send Slack alert" helper
Template	a shared workflow you can copy	"Sheets to Slack digest"

✓ Remember

Workflow → made of nodes → started by a trigger → passes items (JSON) → configured with expressions → logs into apps with credentials → each run is an execution.

☆ Key Takeaway ☆

Learn these 14 words and every n8n tutorial will make sense.

Building blocks

12

Triggers: How Workflows Start

Every workflow starts with **one trigger** (you can have several triggers in one workflow, each starts its own run).

Trigger	Fires when	PeakFit use
Manual (When clicking Execute workflow)	you click the button	testing while building
Schedule	on a timetable	Monday 08:00 report
Webhook	another system calls a URL	website sends a new booking
Form (On form submission)	someone submits an n8n-hosted form	free trial request form
Chat	a message arrives in a chat window	member FAQ bot
App triggers (Gmail, Sheets, Slack…)	something happens in that app	new row in the sign-up sheet
Called by another workflow	a parent workflow calls it	reusable helpers
Error Trigger	another workflow fails	alert the admin

Schedule rules

Setting	Means
Every 15 minutes	96 runs a day
Every day at 07:30	1 run a day
Weeks → Monday → 08:00	weekly report
Custom cron 0 8 * * 1	minute 0, hour 8, any day, any month, Monday
Custom cron */30 9-17 * * 1-5	every 30 min, 09:00 to 17:59, Mon to Fri

Polling

App triggers like "new row in Sheets" **check every few minutes**. Small delay, uses executions only when there's something new.

Webhook (push)

The other app **calls n8n instantly**. Fastest, but your n8n must be reachable from the internet.

Watch out

Check the **timezone** in Workflow Settings. A server in UTC runs "08:00" at 09:00 or 10:00 German time.

Key Takeaway

Pick the trigger by asking "what event should start this?": a time, a click, a call from another app, or a message.

Building blocks

13

How Data Flows: Items

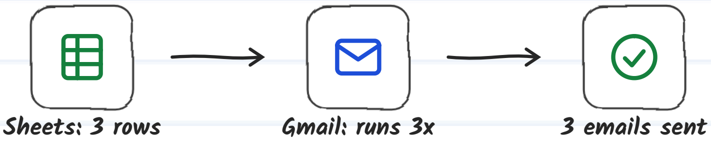
This is the idea that makes everything else click. n8n passes a list of **items**, and most nodes run once **per item**.

The sheet

name	**studio**	**plan**
Lea	Aachen	Basic
Tom	Cologne	Premium
Mia	Aachen	Premium

What n8n sees (JSON)

```
[
  { "json": {
      "name": "Lea",
      "studio": "Aachen",
      "plan": "Basic" } },
  { "json": { "name": "Tom", ... } },
  { "json": { "name": "Mia", ... } }
]
```



3 items go in, the Gmail node sends 3 emails automatically

Rules worth memorising

- **1 row = 1 item.** A sheet with 200 rows → 200 items.
- Each item holds its data under **json** (and files under **binary**).
- A node with 3 items in **runs 3 times** (no loop needed!).
- Want **one** email listing all 3? First combine them with **Aggregate** or **Summarize**.
- Nested data uses dots: **{"address": {"city": "Aachen"}}** → **address.city**.

⚠ Watch out

Classic beginner surprise: "Why did it send 200 emails?" Because 200 items arrived. Check the item count on the connection line before an action node.

☆ Key Takeaway ☆

Think in items, not cells. Each node takes a list of items, does its job per item, and passes a new list on.

14

Expressions: Making Fields Dynamic

A field can hold **fixed text** or an **expression**. Expressions live inside `{{ }}` and are calculated for **each item**.

Switch a field to expression mode

1. **Hover** the field and pick *Expression* (or type `{{`)
2. **Drag** a field from the Input panel into it
3. **Read** the live preview of the result underneath

The 4 references you'll use most

Expression	Means	Result (item = Lea)
`{{ $json.name }}`	field of the **current item**	Lea
`{{ $json["first name"] }}`	field name with a space	Lea
`{{ $('Get sign-ups').item.json.studio }}`	field from an **earlier node**, same item	Aachen
`{{ $('Get sign-ups').first().json.name }}`	first item of an earlier node	Lea
`{{ $input.all().length }}`	number of items coming in	3

Mixing text and expressions

You type	Result
`Hi {{ $json.name }}, welcome to PeakFit {{ $json.studio }}!`	Hi Lea, welcome to PeakFit Aachen!
`Plan: {{ $json.plan.toUpperCase() }}`	Plan: BASIC
`{{ $json.plan === 'Premium' ? 'Free towel' : 'Upgrade?' }}`	Upgrade?

Good to know

Everything inside `{{ }}` is **JavaScript**. You don't need to learn JavaScript first: the cookbook on the next page covers 90% of real needs.

⚠ Watch out

Field names are **case-sensitive**: `$json.Name` is not `$json.name`. If the preview shows *undefined*, the name or the path is wrong.

☆ Key Takeaway ☆

`{{ $json.field }}` for this item, `{{ $('Node').item.json.field }}` for an earlier node. Drag fields instead of typing them.

Building blocks

15 Expressions Cookbook

Text

Goal	Expression	Result
trim + lowercase an email	`{{ $json.email.trim().toLowerCase() }}`	lea@mail.de
first name only	`{{ $json.name.split(' ')[0] }}`	Lea from "Lea Becker"
title case (n8n helper)	`{{ $json.city.toTitleCase() }}`	Aachen
join a list	`{{ $json.tags.join(', ') }}`	yoga, spin
fallback if empty	`{{ $json.phone || 'no phone' }}`	no phone

Numbers

Goal	Expression	Result
add VAT	`{{ $json.price * 1.19 }}`	59.5 for 50
round to 2 decimals	`{{ ($json.price * 1.19).toFixed(2) }}`	59.50
percentage	`{{ Math.round($json.done / $json.total * 100) }}%`	75%

Dates (n8n uses the Luxon library)

Goal	Expression	Result (today = 4 Nov 2026)
today, German format	`{{ $today.toFormat('dd.MM.yyyy') }}`	04.11.2026
7 days ago (ISO)	`{{ $today.minus({days: 7}).toISODate() }}`	2026-10-28
now with time	`{{ $now.toFormat('yyyy-MM-dd HH:mm') }}`	2026-11-04 08:00
weekday name	`{{ $today.toFormat('cccc') }}`	Wednesday
parse a text date	`{{ DateTime.fromISO($json.date).toFormat('dd LLL') }}`	03 Nov
days until a date	`{{ Math.ceil(DateTime.fromISO($json.due).diff($today, 'days').days) }}`	5

Conditions

Goal	Expression
if / else	`{{ $json.amount > 100 ? 'big' : 'small' }}`
check empty	`{{ $json.notes ? $json.notes : '-' }}`

Tip

Stuck? Ask the built-in AI assistant (cloud) or any chatbot: "n8n expression to get the month name from {{ $json.date }}".

☆ **Key Takeaway** ☆

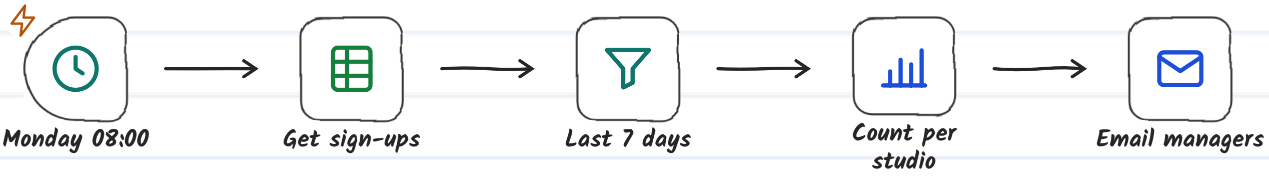
Text, numbers, dates and if/else cover almost everything. **`$today`** and **`$now`** are your date friends.

Building blocks

16

Your First Workflow, Step by Step

Goal: every Monday at 08:00, email the studio managers how many people signed up last week, per studio.



The finished workflow

The sheet "Sign-ups" (tab: 2026)

date	name	studio	plan
2026-10-27	Lea	Aachen	Basic
2026-10-29	Tom	Cologne	Premium
2026-10-31	Mia	Aachen	Premium
2026-11-02	Jonas	Düsseldorf	Basic
2026-10-12	Sara	Cologne	Basic

Build it

1. **New workflow** → name it "Weekly sign-ups". Add **Schedule Trigger**: Weeks, Monday, hour 8.
2. Tab → **Google Sheets → Get row(s) in sheet.** Create a Google credential (sign in with Google), pick the file and tab. Click **Execute step**: 5 items.
3. **Filter** node: condition `{{ DateTime.fromISO($json.date) }}` *is after* `{{ $today.minus({days: 7}) }}` (type: Date & Time). Test on Mon 2 Nov: 4 items pass, Sara (12 Oct) is dropped.
4. **Summarize** node: *Aggregation* = Count, field **name**; *Split By* = **studio**. Output: Aachen 2 · Cologne 1 · Düsseldorf 1.
5. **Gmail → Send a message.** Summarize gave **3 items** (one per studio), so Gmail would send 3 emails. For one email, add an Aggregate first (tip below). Subject: `Sign-ups, week {{ $today.toFormat('W') }}`.
6. **Execute workflow** to test, check the email, then **Publish**.

One email instead of three

Put an **Aggregate** node (Individual fields → **studio**, **count_name**) before Gmail, then write the body as `{{ $json.studio.map((s, i) => s + ': ' + $json.count_name[i]).join('\n') }}`. Result: one email with three lines.

Watch out

Date columns from Sheets often arrive as **text** like 27.10.2026. Then use `DateTime.fromFormat($json.date, 'dd.MM.yyyy')` instead of `fromISO`.

☆ Key Takeaway ☆

Trigger → get data → filter → summarise → send. Most reporting workflows are this exact shape.

Building blocks

17

Core Nodes You'll Use Daily

Node	Does	Example
Edit Fields (Set)	add, rename, remove fields	create **full_name** from first + last
If	true / false split	plan is Premium?
Switch	many routes by rules	route by studio: Aachen / Cologne / Düsseldorf
Filter	keep only matching items, drop the rest	only last 7 days
Merge	combine two streams	sign-ups + payment status
Aggregate	many items → one item with lists	one email listing all names
Summarize	count / sum / average, grouped	sign-ups per studio
Split Out	one item with a list → many items	a list of 5 tags → 5 items
Loop Over Items	process in batches	50 contacts at a time
Wait	pause for time, a date, or a reply	wait 2 days, then follow up
HTTP Request	call any API	weather, CRM, internal tool
Code	JavaScript / Python for custom logic	complex calculations
Remove Duplicates	drop repeated items	same email twice
Sort / Limit	order and cut	top 5 newest
Date & Time	format / shift dates	add 30 days to a start date
Extract from File / Convert to File	read CSV, PDF, Excel / create them	email a CSV export
Respond to Webhook	send an answer back to the caller	"booking received"
Stop and Error	fail on purpose with a message	missing email address
Execute Sub-workflow	call another workflow	shared "notify Slack" helper

Edit Fields in action

Mode	Field name	Value	Result for Lea
Manual	**member_id**	**PF-{{ $json.studio.slice(0,3).toUpperCase() }}-{{ $json.name }}**	**PF-AAC-Lea**
Manual	**monthly_fee**	**{{ $json.plan === 'Premium' ? 49 : 29 }}** (type Number)	**29**
Option	*Include Other Input Fields*	on	keeps name, studio, plan too

Pro move

Name every node by what it does ("Get last week's sign-ups"), not by its type ("Google Sheets1"). Expressions like **$('Get last week sign-ups')** then read like English.

Key Takeaway

About 12 core nodes do most of the work. Learn Edit Fields, If, Filter, Merge, Aggregate and HTTP Request first.

18

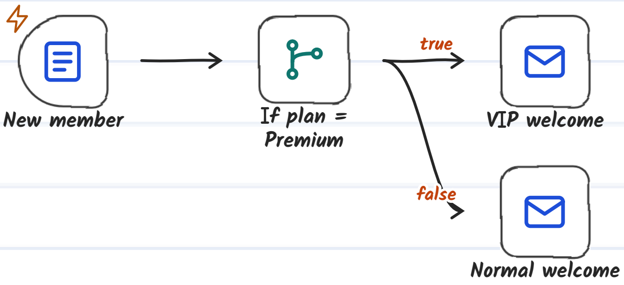
Logic: If, Switch, Filter

If
Two exits: **true** and **false**. Both can continue.

Switch
Many exits, one per rule (plus a fallback).

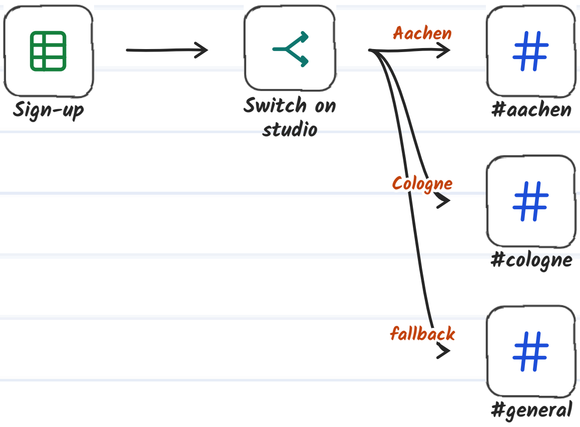
Filter
One exit. Items that fail are **dropped**.

If: Premium members get a different welcome


New member
If plan = Premium
true
false
VIP welcome
Normal welcome

Setting	Value
Value 1	{{ $json.plan }}
Operation	String → *is equal to*
Value 2	Premium
Combine several conditions	**AND** (all must match) or **OR** (any)

Switch: send each sign-up to its studio's Slack channel


Sign-up
Switch on studio
Aachen
Cologne
fallback
#aachen
#cologne
#general

Comparison types you can pick

Type	Operators
String	is equal, contains, starts with, ends with, matches regex, is empty
Number	=, ≠, >, <, ≥, ≤
Date & Time	is after, is before, is equal
Boolean	is true, is false
Array	contains, length equals, is empty

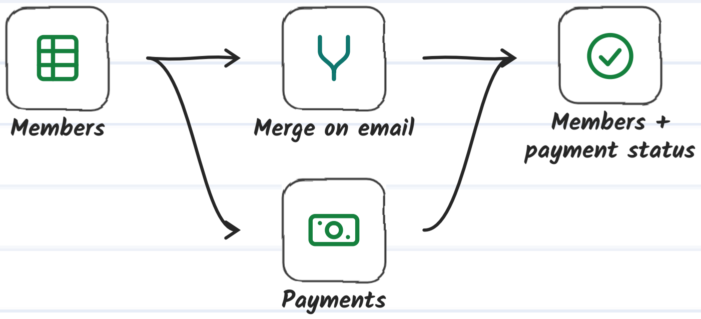
⚠ Watch out
Comparing a number stored as text ("49") with a number (49)? Turn on **Convert types where required** in the node options, or cast with `{{ Number($json.fee) }}`.

☆ **Key Takeaway** ☆
Filter = "only keep these". If = "two paths". Switch = "many paths".

Building blocks

19 Combining Data: Merge, Aggregate, Split Out

Merge: join two streams like a lookup



Like VLOOKUP / a SQL join

Merge mode	Does	Think of it as
Append	stream 1 items, then stream 2 items	stacking two lists
Combine → Matching Fields	pairs items where e.g. **email** matches	VLOOKUP / JOIN
Combine → Position	1st with 1st, 2nd with 2nd	side by side
Combine → All Combinations	every item with every item	cross join (careful!)
SQL Query	write SQL over both inputs	full control
Choose Branch	output one input's data, wait for both	"continue when both are done"

Matching-field joins can **keep matches only**, **keep everything**, or **keep non-matches** (great for "who hasn't paid?").

Aggregate: many → one

3 items in: **{name: Lea}**, **{name: Tom}**, **{name: Mia}**

1 item out: **{ "name": ["Lea", "Tom", "Mia"] }**

Use before sending **one** message about many records.

Split Out: one → many

1 item in: **{ "classes": ["Yoga", "Spin", "HIIT"] }**

3 items out: **{classes: Yoga}**, **{classes: Spin}**, **{classes: HIIT}**

Use when an API returns a list inside one item.

Example

"Who signed up but hasn't paid?" Members + Payments → Merge (Matching Fields on email, keep **non-matches** from input 1) → list of unpaid members → reminder email.

☆ **Key Takeaway** ☆

Merge joins streams, Aggregate squeezes many items into one, Split Out explodes one into many.

Building blocks

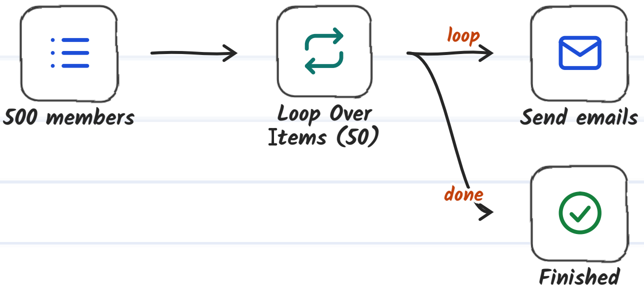
20

Loops, Batches & Waiting

Do I even need a loop?

Usually not. Nodes already run once per item. You need **Loop Over Items** only when:

- an API allows only a few requests at a time (rate limits)
- you must process in **batches** (e.g. 50 emails, pause, next 50)
- a step must finish fully for one item before the next starts

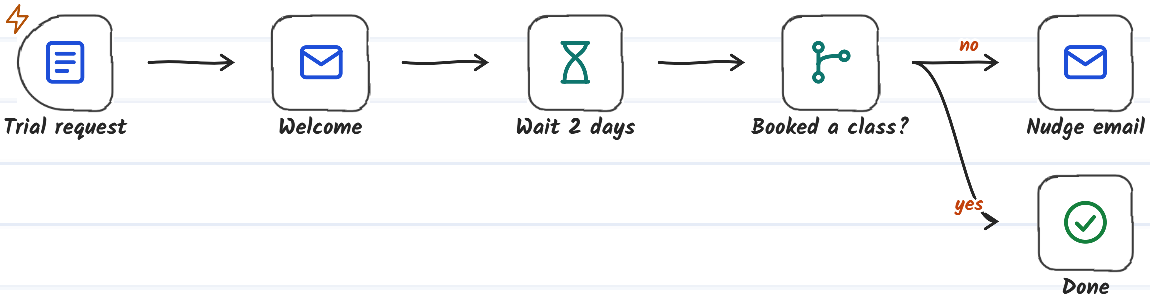


Loop Over Items has two exits: loop and done

The *loop* exit is connected back into Loop Over Items after the Send node (and a short **Wait**), so it repeats until all batches are done.

Wait node: pause the workflow

Resume...	Example
after a time interval	wait 2 days after a trial sign-up, then send a check-in
at a specific time	send at 09:00 on the class day
on a webhook call	wait until the payment provider confirms
on a form submission	manager approves via a form, then continue



A follow-up sequence

Good to know

Long waits (hours, days) are saved to the database, so they survive restarts. Short waits (under about a minute) keep the run in memory.

⚠ Watch out

A loop with no exit runs forever and burns executions. Always test with a **Limit** node (e.g. 3 items) first.

☆ Key Takeaway ☆

Items loop by themselves. Use Loop Over Items for batches and rate limits, Wait for time delays and approvals.

Building blocks

21

Talking to Any App: HTTP Request & APIs

No ready-made node for an app? If it has an **API**, the **HTTP Request** node can talk to it.

API words in plain English

Word	Means
Endpoint / URL	the address you call, e.g. **https://api.example.com/v1/members**
Method	**GET** read · **POST** create · **PUT/PATCH** update · **DELETE** remove
Query parameters	filters in the URL: **?studio=Aachen&limit=10**
Headers	extra info, often the API key: **Authorization: Bearer abc123**
Body	the data you send with POST/PUT, usually JSON
Status code	200 OK · 201 created · 400 bad request · 401 not allowed · 404 not found · 429 too many requests · 500 server error

Example: tomorrow's rain chance in Aachen (free API, no key)

Setting	Value
Method	GET
URL	**https://api.open-meteo.com/v1/forecast**
Query	**latitude=50.78** · **longitude=6.08** · **daily=precipitation_probability_max** · **timezone=Europe/Berlin** · **forecast_days=2**

Response (shortened), one item:

```
{ "daily": { "time": ["2026-11-04", "2026-11-05"],
             "precipitation_probability_max": [20, 70] } }
```

Tomorrow's chance: **{{ $json.daily.precipitation_probability_max[1] }}** → **70**

Power features

- **Import cURL**: paste a **curl** command from the API docs and n8n fills in every field
- **Authentication**: use a saved credential (Header Auth, Bearer, Basic, OAuth2) instead of pasting keys
- **Pagination**: options to follow "next page" automatically
- **Batching / retry**: slow down for rate limits, retry on 429

Tip

Test any API in the node first with **Execute step**. Read the error message: a 401 almost always means the key or header is wrong.

☆ **Key Takeaway** ☆

Endpoint + method + auth + parameters = any API. HTTP Request makes n8n work with practically every modern tool.

Building blocks

22

Webhooks & Forms

Webhook node: give your workflow a URL

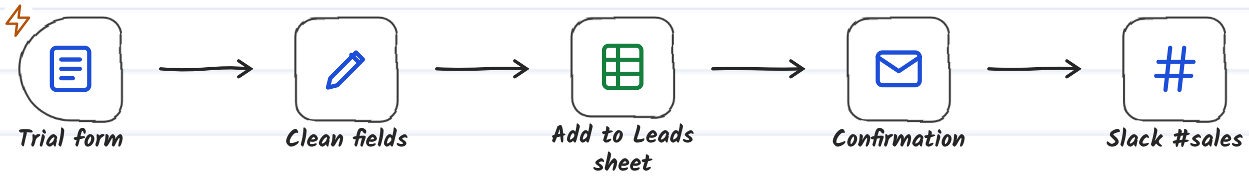
Another system (website, payment provider, booking tool) **sends data to that URL**, and the workflow starts instantly.

	Test URL	Production URL
Works when	you clicked **Listen for test event** (once)	the workflow is **published**
Use for	building and testing	real traffic
Shows data in	the editor	the Executions list

Respond to the caller

Respond option	Use
Immediately	"got it", process afterwards (fast)
When last node finishes	caller waits for the final result
Using Respond to Webhook node	full control of the reply and status code

Form trigger: a form without any website



The PeakFit free-trial form

- Add fields (name, email, studio dropdown, preferred time) in the node, and n8n hosts the page
- Choose the message or redirect shown after submit
- Multi-page forms: add **n8n Form** nodes later in the workflow

Test a webhook from your terminal

```
curl -X POST https://your-n8n/webhook-test/booking -H "Content-Type: application/json" -d
'{"name":"Lea","class":"Yoga","date":"2026-11-06"}'
```

In the workflow: **{{ $json.body.name }}** → **Lea** (webhook data arrives under **body**).

⚠ **Watch out**

Anyone who knows the URL can call it. Add **authentication** in the Webhook node (Header or Basic auth) for anything real, and never put secrets in the URL.

☆ **Key Takeaway** ☆

Webhook = other apps push data in. Form trigger = people fill in a form you didn't have to build.

Building blocks

23

The Code **Node**

For logic that would need 5 nodes, or calculations the built-in nodes can't do. Languages: **JavaScript** *(default) and* **Python***.*

Two modes

Mode	Runs	Use
Run Once for All Items	one time, gets every item	totals, sorting, comparing items
Run Once for Each Item	once per item	change each item on its own

Example 1: add fields to every item (all-items mode)

```
const out = [];
for (const item of $input.all()) {
  const fee = item.json.plan === 'Premium' ? 49 : 29;
  out.push({ json: { ...item.json, monthly_fee: fee, yearly: fee * 12 } });
}
return out;
```

Lea (Basic) → ***monthly_fee: 29, yearly: 348***

Example 2: one summary item

```
const items = $input.all();
const total = items.reduce((sum, i) => sum + i.json.monthly_fee, 0);
return [{ json: { members: items.length, revenue: total } }];
```

For Lea 29 + Tom 49 + Mia 49 → ***{ members: 3, revenue: 127 }***

Rules that save you an hour of debugging

- ***Return a list of items****, each with a* ***json*** *object:* ***return [{ json: {…} }]***
- *Read input with* ***$input.all()*** *(all items) or* ***$json*** *(each-item mode)*
- *Expressions like* ***$('Node Name')*** *and* ***$now*** *also work in code*
- ***console.log(x)*** *prints to the browser console while testing*

Good to know

In n8n 2.0, Code nodes run in a separate **task runner***, so a buggy loop can't crash the whole n8n. Some older tricks (like reading all environment variables) are blocked by default.*

AI angle

Not a coder? Describe the logic to an AI ("n8n Code node, JavaScript, all-items mode: add a field…") and paste the answer. Then test with **Execute step***.*

☆ Key Takeaway ☆

Use the Code node when you'd otherwise chain many nodes. Always return a list of ***{ json: … }*** *items.*

Building blocks

24 Credentials & Security

How credentials work

- Saved **once**, reused by every node (pick from a dropdown)
- Stored **encrypted** in n8n's database
- Can be shared with teammates without showing the secret
- Types: **OAuth2** ("Sign in with Google/Microsoft"), **API key**, **Basic auth**, **Header auth**

1. **Open** a node, pick *Credential → Create new*
2. **Follow** the guide link shown (where to get the key)
3. **Sign in** (OAuth) or paste the key
4. **Test** the connection, then name it ("PeakFit Gmail")

Security habits

✓ Never paste API keys into expressions, Code or sticky notes, only into credentials

✓ Give each app the **least access** it needs (read-only where possible)

✓ Protect webhooks with auth; validate incoming data

✓ Self-hosting: keep **N8N_ENCRYPTION_KEY** safe (without it, saved credentials can't be decrypted after a move)

✓ Use HTTPS and strong passwords; turn on 2FA for your n8n account

✓ Keep n8n updated (security fixes)

☐ Shared test data with real personal data (avoid: use fake data, GDPR!)

⚠ **Watch out**

Exported workflows don't include credential secrets, but they **do** include URLs, sheet IDs and anything you typed into fields. Check before sharing a workflow publicly.

☆ **Key Takeaway** ☆

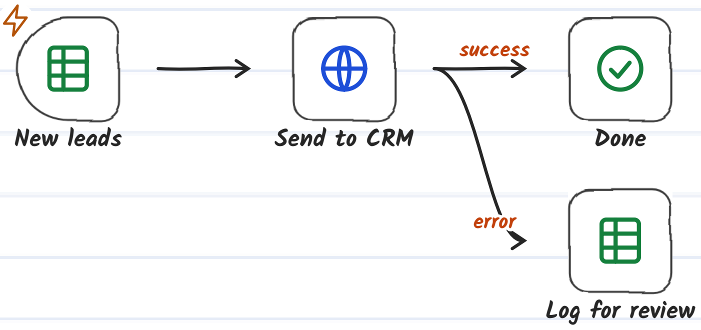
Secrets go in credentials, never in fields. Least access, protected webhooks, updated n8n.

25 When Things Break: Error Handling

APIs time out, a sheet column gets renamed, an email is invalid. Plan for it.

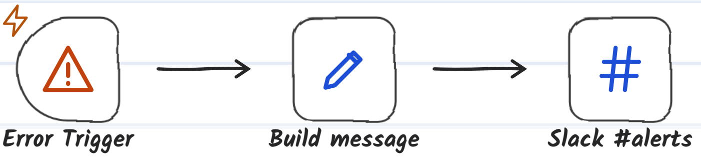
Layer 1: node settings

Setting	Effect
Retry On Fail	tries again (set Max Tries, e.g. 3, and Wait Between Tries, e.g. 5000 ms)
On Error → Stop Workflow	default: run fails, shown red in Executions
On Error → Continue	ignores the error, passes on
On Error → Continue (using error output)	adds a red **error** exit you can route to a fallback



Error output: bad rows go to a review sheet instead of stopping everything

Layer 2: an error workflow (alerts)



Separate workflow, set in the main workflow's Settings → Error workflow

The Error Trigger receives the failed workflow's name, the node that broke, the error message and a link to the execution.

Layer 3: fail on purpose

Stop and Error node: stop with your own message when data is wrong, e.g. `Missing email for {{ $json.name }}`.

☆ **Pro move**

Make one "Error alerts" workflow and set it as the error workflow on **every** production workflow. You'll hear about problems before your users do.

☆ **Key Takeaway** ☆

Retry for hiccups, error outputs for bad records, an error workflow for alerts.

Building blocks

26 Debugging Like a Pro

Tool	How	Why
Execute step	button in a node	test one node without running everything
Pin data	pin icon on a node's output (or P)	freeze sample data, stop calling the API every test
Edit output	edit pinned data by hand	simulate edge cases ("what if email is empty?")
Executions list	Executions tab	see every past run, the data at each node, and where it failed
Debug in editor	in a failed execution	load that run's data into the editor to fix it (availability depends on plan/version)
Disable node	select node, D	skip a step while testing
Limit node	add before expensive steps	test with 2 items, not 2,000
Sticky notes	Shift + S	leave explanations for future you

A debugging routine

1. **Find** the first red node in the execution
2. **Read** its error and its INPUT data
3. **Pin** that input and fix the node
4. **Re-run** the step, then the workflow

Common errors and fixes

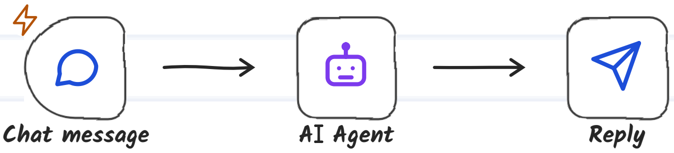
You see	Usually means	Fix
Cannot read properties of undefined	a field name/path is wrong or empty	check the exact name in INPUT, add a fallback
401 / 403	credential or permission problem	reconnect the credential, check scopes
404	wrong URL or ID	copy the ID again from the app
429	too many requests	Retry On Fail with waits, Loop Over Items with Wait
workflow ran but did nothing	0 items reached the node	check Filter / If conditions, look at item counts
works in test, not live	not published, or test vs production URL	Publish, use the production webhook URL

☆ **Key Takeaway** ☆

Pin data, test one step at a time, and read the INPUT panel of the first red node. That solves most problems.

27 AI in n8n: The Building Blocks

n8n's AI features are normal nodes. An **AI Agent** is a "root" node with smaller **sub-nodes** plugged into it.



An AI agent and its plug-ins

Chat model

required. The LLM that thinks: OpenAI, Anthropic, Google Gemini, Mistral, local Ollama...

Memory

optional. Remembers the conversation (e.g. last 10 messages).

Tools

optional. Things the agent may **do**: search, call an API, read a sheet, run a sub-workflow.

Output parser

optional. Forces a clean structure (JSON fields) for the next nodes.

Which AI node do I need?

Task	Simplest node
one prompt in, one answer out (summarise, rewrite)	**Basic LLM Chain** (or the model's own node, e.g. OpenAI "Message a model")
put text into categories (support / sales / spam)	**Text Classifier**
pull fields out of text (name, date, amount)	**Information Extractor**
positive / negative / neutral	**Sentiment Analysis**
summarise long documents	**Summarization Chain**
decide what to do and use tools, multi-step	**AI Agent**
answer questions from your own documents (RAG)	AI Agent + **vector store** (Pinecone, Qdrant, Supabase, in-memory) + embeddings

Words you'll meet

Word	Plain English
LLM	the AI model that reads and writes text
Prompt / system message	instructions: who the agent is and what rules it follows
Tool	an action the agent may call by itself
RAG	look up relevant snippets in your documents, then answer with them
Embeddings / vector store	documents turned into numbers so similar meaning can be searched
Tokens	the units AI providers bill by (roughly 3/4 of a word)

€ Cost check

AI model calls are billed by the AI provider (your API key), separate from n8n executions. A short classification is usually a fraction of a cent; long documents and big models cost more.

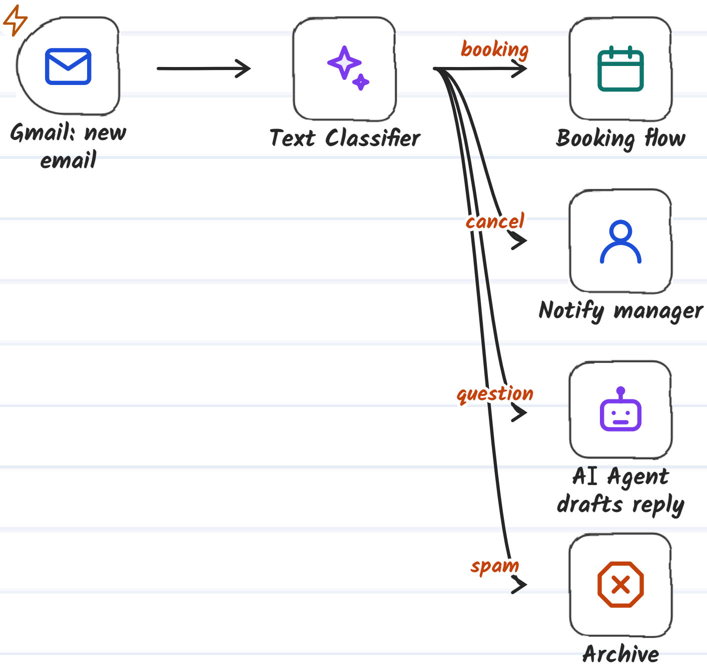
☆ Key Takeaway ☆

Start with the simple AI nodes (Classifier, Extractor, LLM Chain). Use an AI Agent when the AI must decide which tools to use.

AI & advanced

28 Build an AI Inbox Assistant

Goal: PeakFit's info@ inbox gets 60 emails a day. Sort them automatically and draft replies.



Gmail: new email
Text Classifier
booking
Booking flow
cancel
Notify manager
question
AI Agent drafts reply
spam
Archive

Step by step

1. **Gmail Trigger** (On message received), poll every minute, *Simplify* on.
2. **Text Classifier**: input `{{ $json.subject }} {{ $json.text }}`. Categories with a one-line description each:
   - *booking*: wants to book or change a class
   - *cancel*: wants to cancel a membership
   - *question*: general questions (prices, opening hours)
   - *spam*: ads, unrelated offers
3. Connect a **chat model** sub-node (e.g. a small, cheap model is plenty for sorting).
4. On the *question* branch: **AI Agent** with this system message:

> **System message**
>
> You are PeakFit's friendly front desk. Answer in the language of the email. Use only facts from the FAQ tool. If unsure, say a colleague will reply within 24 hours. Never promise refunds.

5. Give the agent a **tool**: a Google Sheets tool reading the FAQ sheet (prices, hours, studios).
6. **Gmail → Create draft** (not send!) with the agent's answer. A human checks and clicks send.

☆ **Key Takeaway** ☆

Gmail trigger → Classifier → one branch per category. Draft, don't send.

AI & advanced

29 Inbox Assistant: Test & Tune

Example run

Incoming email	Category	What happens
"Kann ich morgen um 18 Uhr Spinning buchen?"	booking	booking flow
"What are your opening hours in Cologne?"	question	draft: "Hi! Our Cologne studio is open Mon to Fri 06:00 to 22:00..."
"Bitte kündigt meinen Vertrag"	cancel	manager notified in Slack
"Cheap SEO services!!!"	spam	archived

⚠ Keep a human in the loop

Let AI **draft**, not send, until you've checked a few weeks of results. Log every classification to a sheet so you can spot mistakes.

Tune it before trusting it

- ✓ Run it on **50 old emails** with pinned data and count wrong categories
- ✓ Improve the category **descriptions** first (cheaper than a bigger model)
- ✓ Add an **"other"** category so odd emails don't get forced into a wrong box
- ✓ Log subject, category and date to a sheet for a weekly check
- ✓ Only then switch the question branch from *draft* to *send* (if ever)

☆ Key Takeaway ☆

Classifier to sort, Agent with tools to draft, a human to approve. That's a safe, useful first AI workflow.

30 AI Builder, Assistant & MCP

Let AI build the workflow for you

AI Workflow Builder

Describe what you want in plain language ("Every Monday, read my sign-up sheet and email a per-studio summary"). It picks nodes, places and configures them; you refine with follow-up prompts.

- Each building message uses **one credit** from your plan's monthly allowance
- Your prompt and workflow structure are sent to the AI; **credentials are not**
- Always review parameters and credentials it set

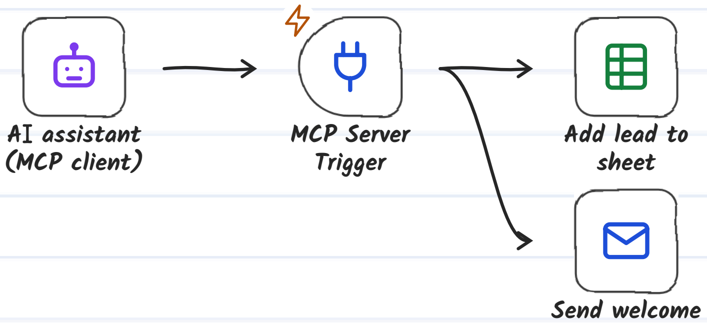
AI Assistant

Built-in helper in n8n Cloud: explains errors, suggests fixes, answers "how do I..." questions about nodes and expressions. Plans include a monthly number of assistant credits.

MCP: connect n8n with AI apps

MCP (Model Context Protocol) is a standard way for AI assistants to use outside tools.

Node	Direction	Example
MCP Server Trigger	your n8n workflows become **tools** that AI apps (e.g. Claude Desktop, other MCP clients) can call	"Hey assistant, add this lead to PeakFit's CRM" runs your n8n workflow
MCP Client Tool	your n8n **AI Agent** uses tools from an external MCP server	the agent can use a company's MCP server for docs or tickets



MCP Server Trigger: an outside assistant calls your workflow as a tool

Good to know

n8n also added instance-level MCP access and prompt-based workflow building during 2026. Features move fast: check the docs and release notes for what your version and plan include.

☆ Key Takeaway ☆

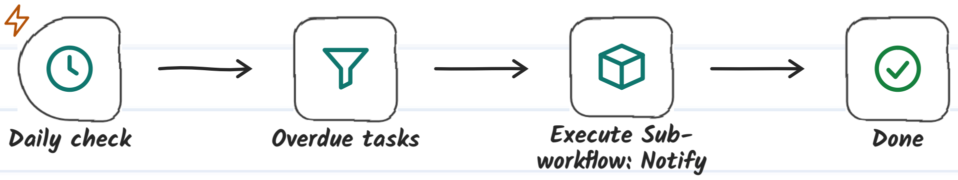
Builder = AI makes workflows. Assistant = AI helps you fix them. MCP = AI apps and n8n use each other's tools.

Automate the boring parts, keep the thinking parts.

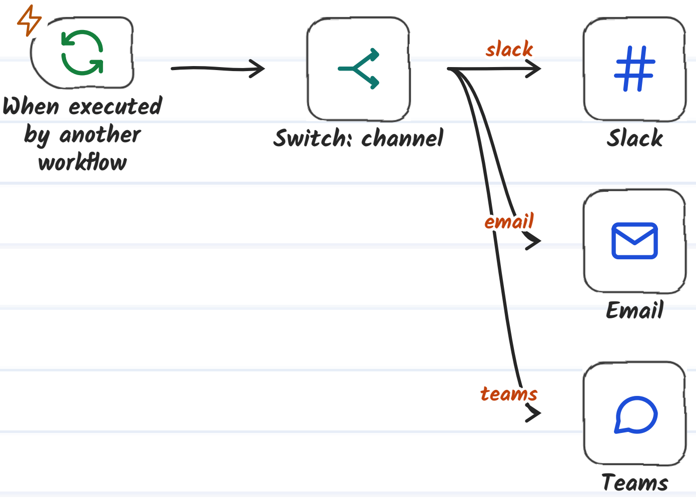
AI & advanced

31 Sub-workflows: Build Once, Reuse Everywhere

Like functions in Excel or code: one small workflow that many others call.


Daily check
Overdue tasks
Execute Sub-workflow: Notify
Done

Main workflow calls a shared helper


When executed by another workflow
Switch: channel
slack
email
teams
Slack
Email
Teams

The helper (its own workflow)

How to set it up

1. **Create** the helper with trigger "When Executed by Another Workflow"
2. **Define** its inputs (e.g. **channel**, **message**, **owner**)
3. **Publish** the helper (needed in n8n 2.0)
4. **Call** it from any workflow with Execute Sub-workflow

Why bother?

- Fix a bug **once**, every workflow benefits
- Big workflows stay readable (each piece does one job)
- Sub-workflows can also be **tools** for an AI Agent ("Call n8n Workflow Tool")
- In n8n 2.0 the parent **waits for the result**, even if the helper pauses on a Wait node (e.g. an approval)

> ☆ **Pro move**
>
> Good first helpers: "Notify" (Slack/email/Teams), "Log to sheet", "Get member by email", "Error alert".

☆ **Key Takeaway** ☆

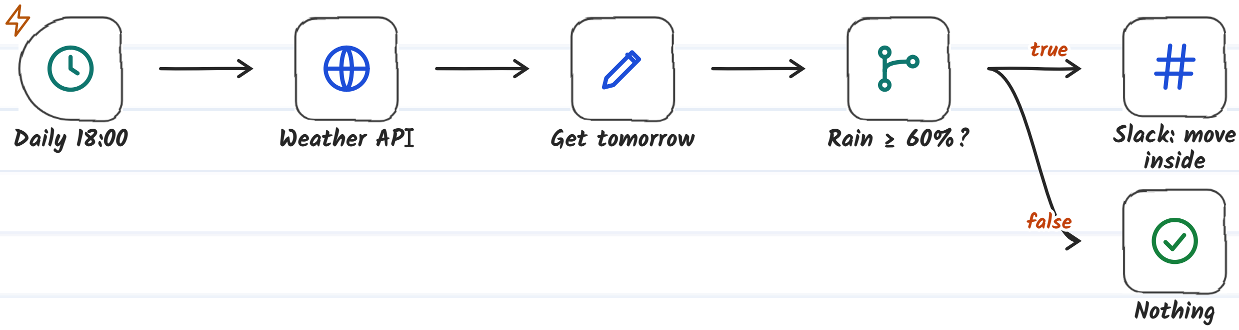
Repeated steps belong in a sub-workflow. Publish it, define inputs, call it everywhere.

Automate the boring parts, keep the thinking parts.

32

Project 1: Rain Alert for Outdoor Classes

PeakFit Aachen runs an outdoor bootcamp every morning. If rain is likely, move it inside and tell members.



Node by node

#	Node	Settings
1	Schedule Trigger	every day, 18:00, timezone Europe/Berlin
2	HTTP Request	GET Open-Meteo forecast for Aachen (see the API page), `forecast_days=2`
3	Edit Fields	`rain = {{ $json.daily.precipitation_probability_max[1] }}` (number) · `day = {{ $json.daily.time[1] }}`
4	If	`{{ $json.rain }}` *is greater than or equal to* **60**
5	Slack (true)	channel #aachen-members: `Rain chance {{ $json.rain }}% on {{ DateTime.fromISO($json.day).toFormat('cccc dd.MM.') }}: tomorrow's bootcamp moves to Studio 2.`

Test result (forecast 20%, 70%): rain = 70, If → true, message:

> Rain chance 70% on Thursday 05.11.: tomorrow's bootcamp moves to Studio 2.

Upgrades once it works

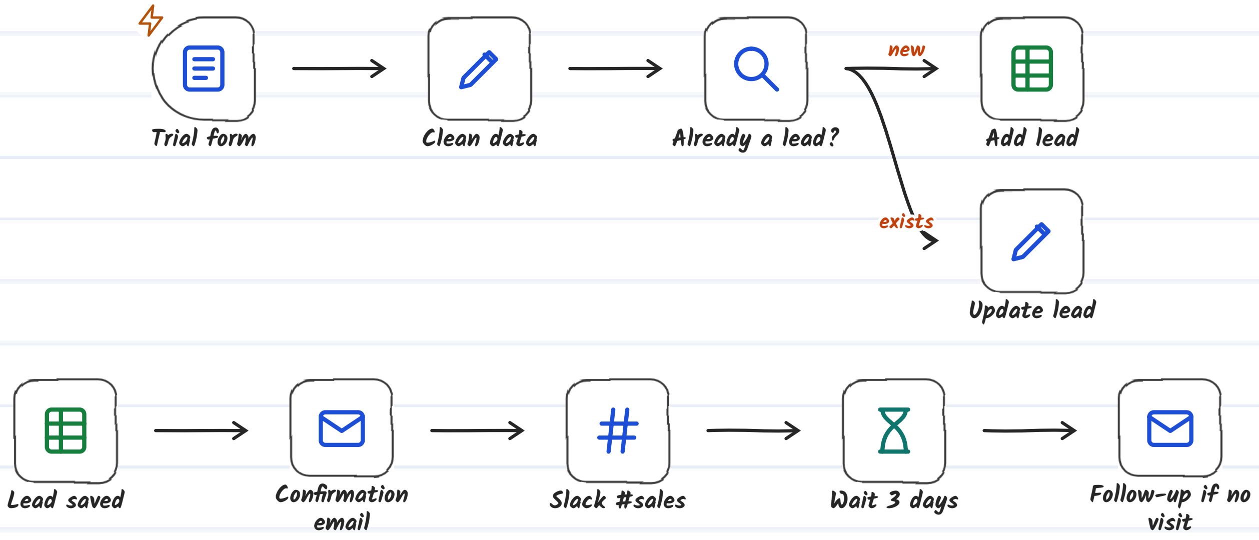
- ☑ also email members booked for that class (Sheets lookup + Gmail)
- ☑ log every check to a sheet (date, rain %, decision) for a season report
- ☐ add wind speed and temperature with a Switch: rain / cold / perfect

☆ **Key Takeaway** ☆

Schedule + HTTP Request + If + message: the most common "watch something and alert me" pattern.

33 Project 2: Trial Form to CRM

From website form to a followed-up lead, with no copy-paste.


Trial form
Clean data
Already a lead?
new
Add lead
exists
Update lead
Lead saved
Confirmation email
Slack #sales
Wait 3 days
Follow-up if no visit

Node by node

Step	Node	Key settings
Form	n8n Form Trigger	fields: name, email, phone, studio (dropdown), goal (dropdown)
Clean	Edit Fields	email = {{ $json.email.trim().toLowerCase() }} · name = {{ $json.name.trim() }} · created = {{ $now.toISO() }}
Check	Google Sheets: Get rows, filter email	then If: found a row?
Save	Sheets: Append (new) / Update (existing)	match column: email
Confirm	Gmail	Hi {{ $json.name }}, your free trial at PeakFit {{ $json.studio }} is booked!
Notify	Slack	New lead: {{ $json.name }} ({{ $json.goal }}) for {{ $json.studio }}
Follow-up	Wait 3 days → Sheets lookup "visited?" → If no → Gmail	friendly reminder with a booking link

Tip

Google Sheets has an **Append or Update** operation: it checks the email column and does the right thing in one node.

Same pattern, other tools

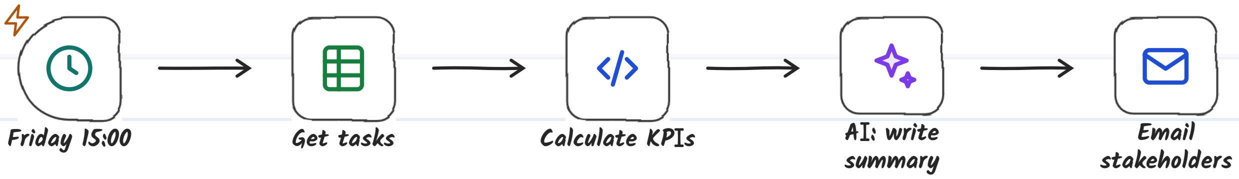
Swap Sheets for HubSpot, Pipedrive or Airtable nodes, swap Slack for Teams. The logic stays identical.

☆ Key Takeaway ☆

Form → clean → de-duplicate → save → confirm → notify → follow up. That's a complete lead pipeline.

Project 3: Weekly PMO Status Report with AI

The PeakFit renovation project tracks tasks in a sheet. Every Friday at 15:00, stakeholders get a short, readable status email.



The tracker (sheet "Tasks")

task	owner	due	status
Order new spin bikes	Omar	2026-10-30	Done
Electrician quote	Sara	2026-11-02	In progress
Repaint studio 2	Tom	2026-11-12	In progress
Update website photos	Lena	2026-11-20	Not started

Code node (all items): KPIs for "today" = Fri 6 Nov 2026

```
const today = DateTime.fromISO('2026-11-06');   // use $today in real life
const t = $input.all().map(i => i.json);
const open = t.filter(x => x.status !== 'Done');
const overdue = open.filter(x => DateTime.fromISO(x.due) < today);
return [{ json: {
  total: t.length, done: t.length - open.length,
  overdue: overdue.map(x => `${x.task} (${x.owner})`),
  pct_done: Math.round((t.length - open.length) / t.length * 100) } }];
```

Result: **total 4 · done 1 · pct_done 25 · overdue: ["Electrician quote (Sara)"]**

AI node (Basic LLM Chain) prompt

Prompt

Write a 5-line project status for busy managers. Use RAG status (Green/Amber/Red). Data: {{ JSON.stringify($json) }}. Be factual, no invented numbers.

Sample email it produces

Status: AMBER

1 of 4 tasks done (25%). Overdue: electrician quote (Sara, due 2 Nov). Next: repaint studio 2 by 12 Nov. Action: Sara to confirm the quote date by Monday.

☆ Key Takeaway ☆

Let code do the counting (exact), let AI do the wording (readable). Never let AI calculate the numbers.

Automate the boring parts, keep the thinking parts.

35 Self-Hosting Without Fear

For workflows that must run 24/7 without paying for Cloud.

A typical small setup

Piece	Why
a small cloud server (VPS), 2 GB RAM	always on
Docker Compose	runs n8n (and the database) with one config file
PostgreSQL database	more robust than the default SQLite for real use
a domain + HTTPS (reverse proxy like Caddy, Traefik or nginx)	secure access, webhooks reachable from the internet
backups	workflows, credentials, database

Environment settings you'll meet

Variable	Example	Why
N8N_HOST / WEBHOOK_URL	n8n.peakfit.de / https://n8n.peakfit.de/	correct webhook URLs
GENERIC_TIMEZONE	Europe/Berlin	schedules run at German time
N8N_ENCRYPTION_KEY	a long random string	encrypts credentials; **back it up**
DB_TYPE	postgresdb	use Postgres

Maintenance checklist

- ✓ Update regularly: pull the new image, restart, read release notes first (major versions can have breaking changes, like 1.x → 2.0)
- ✓ Back up the database **and** the encryption key
- ✓ Export workflows now and then: `n8n export:workflow --all --output=backup/`
- ✓ Watch disk space (execution history grows: set pruning/retention)
- ✓ Use the error workflow so failures reach you

⚠ Watch out

Self-hosting means ***you*** handle security patches and downtime. For company-critical workflows without an IT person, Cloud is often the cheaper choice once you count your time.

☆ Key Takeaway ☆

VPS + Docker Compose + Postgres + HTTPS + backups = a solid self-hosted n8n.

Going further

36

Best Practices for Clean Workflows

Build

- ✓ ***One workflow, one job.*** "Weekly sign-up report", not "Everything Monday"
- ✓ ***Name nodes*** by purpose: "Get overdue tasks", not "Google Sheets3"
- ✓ ***Sticky notes*** on anything clever (Shift+S)
- ✓ Keep secrets in ***credentials***, settings like emails or thresholds in one **Edit Fields** node at the start ("Config")
- ✓ Prefer built-in nodes; use Code when it replaces 5+ nodes

Test

- ✓ Pin sample data, include ***edge cases*** (empty field, weird characters, 0 items)
- ✓ Test with a ***Limit*** node before running on 2,000 records
- ✓ Send test emails to ***yourself*** first

Run

- ✓ Error workflow on every production workflow
- ✓ **Idempotent** design: running twice shouldn't create duplicates (check "already exists?" before creating)
- ✓ Respect rate limits (batches + Wait)
- ✓ Mind execution counts on Cloud: does it really need to run every minute?
- ✓ Use ***sub-workflows*** for repeated parts

Organise

Habit	How
naming convention	**[Area] What it does → [Sales] Trial form to CRM**
tags	sales, reporting, ai, test
versions	workflow history (paid plans) or export JSON to Git
documentation	a sticky note at the start: purpose, owner, trigger, what breaks it

☆ **Key Takeaway** ☆

Small, named, documented, tested workflows with error alerts. Future you will be grateful.

Going further

37 Shortcuts & Speed Tips

Keys	Does
Tab	open the nodes panel
Ctrl + Enter	execute the workflow
Ctrl + S	save
Shift + S	add a sticky note
F2	rename the selected node
D	disable / enable the selected node
P	pin / unpin the node's output data
Ctrl + C / Ctrl + V	copy / paste nodes (even between workflows!)
Delete	delete selected nodes
1	zoom to fit the whole workflow
0	reset zoom
Ctrl + Z	undo

Good to know

Shortcuts can change between versions. Check the keyboard shortcuts list in n8n's Help menu for yours.

Speed tricks

- **Copy-paste nodes as JSON**: copied nodes are JSON text, so you can paste them into a chat, a doc, or another n8n
- **Drag fields** from INPUT instead of typing expressions
- **Templates**: start from a community template, then adapt
- **Import cURL** in HTTP Request
- **Duplicate workflows** to make variants ("Report Aachen" → "Report Cologne")
- **Pinned data** = no waiting for slow APIs while building

☆ **Key Takeaway** ☆

Tab, Ctrl+Enter, Shift+S, D and P will make you noticeably faster on the canvas.

Going further

38 Your 30-Day Learning Plan

Week	Focus	Build this
1	install, editor, items, expressions	Schedule → HTTP Request (weather) → email yourself
2	Sheets, If/Switch/Filter, Merge, Aggregate	the weekly sign-up report (first workflow page)
3	webhooks, forms, error handling, sub-workflows	trial form → CRM pipeline with error alerts
4	AI nodes, agents, MCP	AI inbox assistant or PMO status report

Free learning resources

- **n8n docs** (docs.n8n.io): official guides, node reference, and free text courses (Level 1 and Level 2)
- **Templates** gallery on n8n.io: thousands of example workflows
- **Community forum** (community.n8n.io): search your error message first
- **n8n's YouTube channel**: beginner series and feature updates

Make it portfolio-ready

1. **Pick** a real problem (your job search tracker, a weekly report)
2. **Build** it, measure time saved
3. **Screenshot** the workflow + write a 5-line README
4. **Share** on GitHub / LinkedIn (remove credentials and personal data)

Interview line

"I built an n8n workflow that turns our project tracker into a weekly AI-written status email. It saves the PMO about 2 hours a week."

☆ **Key Takeaway** ☆

One small workflow per week. After 4 weeks you'll understand 80% of what n8n can do, with proof.

Review

39 Q&A: Doubts You'll Probably Have (1)

Is n8n really free?

The self-hosted Community Edition is free to use for yourself and your company. You pay for the server and AI usage. n8n Cloud is paid (from about €20/month on annual billing).

Do I need to know how to code?

No. Most workflows need zero code. Knowing a little about JSON and simple expressions helps a lot, and the Code node is optional.

What's the difference between n8n Cloud and self-hosted?

Same editor. Cloud: n8n runs it, updates it and gives you a public URL. Self-hosted: free licence, you run and maintain it.

What exactly counts as an execution?

One full run of a workflow, whatever the number of steps. A test run you click also shows up, so test with pinned data where possible.

Why didn't my schedule run?

The workflow isn't published (or active in older versions), the laptop/server was off, or the timezone is different from what you expected.

Why did my workflow send 50 emails instead of 1?

50 items reached the email node. Use Aggregate or Summarize first, or Execute Once in the node settings if you only need the first item.

Can n8n read and write Excel files?

Yes: Microsoft Excel 365 / OneDrive nodes for online files, and Extract from File / Convert to File for .xlsx and .csv attachments.

What happens to running workflows when I update n8n?

Short runs finish or fail; long waits are stored and continue after restart. Update when nothing important is running, and read the release notes first.

Review

40 Q&A: Doubts You'll Probably Have (2)

n8n vs Power Automate: which one should I learn?

If your company is all-in on Microsoft 365, Power Automate gets used daily there. n8n is more flexible and cheaper for complex logic and AI. The concepts transfer, so learning one makes the other easy.

Is my data safe?

Self-hosted: data stays on your server. Cloud: hosted by n8n (EU data centres at the time of writing). Either way, data you send to an app or AI model goes to that provider too, so check their policies and avoid sending personal data you don't need to (GDPR).

Test URL vs production URL: why two?

The test URL only listens while you click "Listen for test event" and shows data in the editor. The production URL works when the workflow is published.

Should I use the AI Agent for everything?

No. Agents are slower, cost tokens and can vary. Use normal nodes for exact logic and maths, and AI for language tasks: sorting, extracting, summarising, drafting.

My expression shows "undefined".

The field name or path is wrong (case-sensitive!), or the field is empty for this item. Drag the field from the INPUT panel to get the exact path.

How do I move a workflow to another n8n?

Download it as JSON (or copy all nodes and paste), import it on the other instance, then reconnect the credentials (they're never included).

Can I sell automations I build with n8n?

Yes, building, consulting and maintenance are allowed. Not allowed: offering n8n itself as a service where your customers build workflows (needs a commercial licence).

What should I learn after these notes?

APIs in more depth (auth, pagination), databases (Postgres) as workflow storage, RAG agents with vector stores, and self-hosting with Docker Compose.

☆ **Key Takeaway** ☆

Most n8n problems are about items (how many?), field paths (exact names?) or publishing (is it live?). Check those three first.